Problem Statement 1: Smart Data Masking System for PII Protection

Problem Statement

Financial institutions process millions of sensitive customer records daily across multi-database environments. Ingesting raw Personally Identifiable Information (PII)—such as Credit Card Numbers, Social Security Numbers, and Account Balances—without dynamic column-level access controls exposes sensitive data to unauthorized internal users and creates severe regulatory compliance risks under standards like GDPR and HIPAA. The core challenge is to build an automated, database-agnostic security system that can automatically identify sensitive fields and dynamically mask them at query execution time based on the querying user's access permissions, without altering the underlying raw data.

Student Goal

Design and develop a lightweight data protection engine that connects to a target database, scans schema metadata and row samples to automatically auto-tag PII fields, and enforces role-based dynamic data masking (e.g., Redaction, Partial Masking, Hashing) whenever a query is executed by a specific user role.

Expected Outcome

Automated PII Scanner: A background worker/module that scans column headers and data samples using pattern matching (Regex/rules) to tag sensitive fields with tags like PII_HIGH or FINANCIAL_CONFIDENTIAL.

Dynamic Masking Engine: A database abstraction layer (compatible with at least two database backends, e.g., PostgreSQL, SQLite, MySQL) that intercepts queries or applies view logic to dynamically alter returned values based on user roles (Admin gets full view; Support gets partial masking like 4111-XXXX-XXXX-1111; Analyst gets full redaction or hashes).

Demonstration Mechanism: An interactive Interface (CLI tool, API Postman suite, or low-code Streamlit dashboard) where evaluators can toggle user roles and verify that query results reflect the correct masking policies in real time while leaving underlying database tables intact.